

CENTRALE
LYON

MOD 7.2 : Introduction à la Data Science
BE 1

Prédiction de l'utilisateur d'un logiciel à partir d'un jeu de traces d'utilisation

Élèves :
Taha Zaoui
Oussama Shit
Ndeye Penda Diene

Enseignants :
Dr. Julien Velcin

Table des matières

1 Introduction **2**

2 Chargement et exploration initiale **2**

3 Features engineering **4**

4 Développement d'une solution de classification automatique : **6**

5 Métriques d'Évaluation **7**

6 Résultats des Modèles **8**

7 Amélioration de la solution **8**

8 Identification des Variables Influentes **9**

9 Discussion **9**

10 Conclusion **9**

1 Introduction

Le logiciel Copilote, édité par la société Infologic, concerne le secteur de l'agroalimentaire. Chaque interaction d'un utilisateur avec le logiciel génère une trace d'utilisation enregistrée sous forme de log. Ces traces, qui représentent un volume important de données, constituent une source d'information précieuse : elles permettent d'analyser les comportements d'usage, d'identifier des besoins d'amélioration et d'optimiser les interfaces ou fonctionnalités.

On cherche à exploiter ces traces d'utilisation afin de construire un modèle d'analyse automatique capable de détecter et de classifier les comportements des utilisateurs. L'objectif est de fournir à Infologic des indicateurs pertinents sur les pratiques réelles de ses clients et d'aider à la prise de décision pour l'évolution du logiciel.

La démarche adoptée suit un processus standard d'analyse de données :

1. Exploration des données afin de comprendre leur structure et identifier les variables pertinentes.
2. Construction de caractéristiques (features) adaptées au problème étudié.
3. Mise en œuvre d'algorithmes de classification pour modéliser les comportements.
4. Évaluation et interprétation des résultats à l'aide de métriques de performance.

Cette approche méthodique permet de combiner rigueur statistique et pertinence métier pour valoriser les traces d'utilisation collectées.

2 Chargement et exploration initiale

La première étape du projet consiste à charger les données brutes issues des journaux d'utilisation du logiciel Copilote afin de les examiner et d'en comprendre la structure. Deux approches sont possibles en Python :

— Utiliser la bibliothèque Pandas avec la fonction readcsv() pour charger les données directement dans un dataframe, ce qui permet de bénéficier de fonctionnalités d'analyse puissante.

— Lire le fichier manuellement avec la méthode open() et readlines() pour parcourir les lignes une par une.

On choisit la seconde approche, plus robuste pour ce jeu de données, car les fichiers d'entrées sont irréguliers : certaines lignes contiennent beaucoup plus d'actions que d'autres, et les séparateurs ne sont pas toujours alignés. Une lecture ligne à ligne permet donc est nécessaire.

Première analyse

Après le chargement des données, pour le jeu d'entraînement (featurestrain) on obtient un tableau avec comme colonnes principales

— ***util*** qui est l'identifiant de l'utilisateur (variable cible).

— ***navigateur*** qui est le type de navigateur utilisé (Chrome, Edge, etc.).

Et comme colonnes suivantes : suite d'actions réalisées au cours de la session.

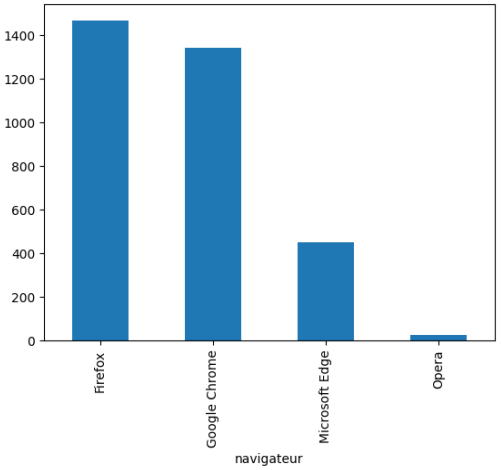
Pour le jeu de test (featurestest), seule la colonne navigateur est présente comme colonne principale, car la variable cible *util* est absente.

La première observation est que le dataFrame d'entraînement contient un nombre considérable de colonne qui est de 14 470 colonnes environ, sachant que la plupart d'entre elles semblent contenir des valeurs NaN. Ce qui peut s'expliquer par le fait que le bibliotheque Pandas étend les lignes comportant moins de colonnes afin qu'elles aient toutes le même nombre de colonne. Ainsi, il complète automatiquement les sessions plus courtes en ajoutant des valeurs NaN afin d'harmoniser la taille des lignes.

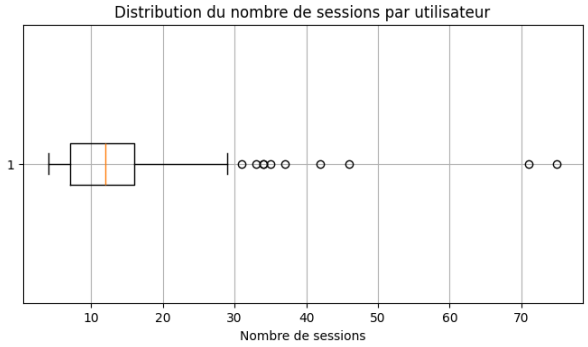
De plus, certaines sessions comportent des marqueurs temporels de type *'txxx'*, représentant des intervalles de 5 secondes entre des actions.

Analyse Exploratoire

Avant d'entamer le prétraitement, on examine la répartition des navigateurs utilisés par utilisateur, ainsi que la répartition du nombre de sessions. Cette étape préliminaire permet de mieux comprendre comment les données sont distribuées et de vérifier si certains utilisateurs dominent le jeu d'entraînement.



(a) Navigateurs



(b) Distribution

FIGURE 1 – Statistiques sur la population.

On observe un déséquilibre des classes où certains modules étant sur-représentés par rapport à d'autres. La majorité des utilisateurs ont entre 5 et 20 sessions, avec une médiane autour de 10 sessions. Cependant, quelques points situés à l'extrême droite du boxplot représentent des valeurs aberrantes : certains utilisateurs possèdent plus de 40, voire 70 sessions.

Ce constat met en évidence un déséquilibre du dataset :

— certains utilisateurs sont surreprésentés, ce qui peut biaiser l'apprentissage ;
— d'autres sont sous-représentés, ce qui rend leurs comportements plus difficiles à modéliser.

Après analyse , on décide de considérer ces cas extrêmes comme de véritables valeurs aberrantes, mais de ne pas les éliminer. Même si leur présence est faible, ces utilisateurs atypiques peuvent apporter des informations importantes sur des comportements rares mais réels, et aider le modèle à mieux généraliser face à des profils variés. Ce choix

permet de préserver la richesse comportementale du jeu de données sans compromettre la stabilité de l'apprentissage.

3 Features engineering

La partie fondamentale du projet est le features engineering. Il s'agit de bien définir les attributs qui seront utilisés par la suite pour entraîner le modèle d'apprentissage supervisé. Dans notre cas, il faut définir des attributs qui permettent de comprendre le comportement de chaque utilisateur devant son écran. Dans notre cas, on propose deux catégories de features :

1. **Des attributs statiques :** qui permettent de comprendre des habitudes adoptées par l'utilisateur dans un contexte global. Cette classe d'attributs compte par exemple :
   - Le navigateur utilisé : cet attribut peut faire la différence entre les utilisateurs ; car deux personnes peuvent avoir des comportements identiques mais utiliser des navigateurs différents.
   - Le temps qu'il met dans la session : des utilisateurs ouvrent de longues sessions alors que d'autres n'utilisent le logiciel que pendant des durées courtes.
   - Le nombre d'actions qu'il effectue au total dans la session .
   - Le nombre de fois qu'il effectue une action donnée (par exemple : entrée en saisie dans un formulaire, clic long, erreur système grave, etc.). Cet attribut fait aussi la différence entre les utilisateurs, par exemple : une personne qui utilise beaucoup le logiciel est censée commettre moins d'erreurs graves qu'un utilisateur débutant.
   - Les écrans, les configurations d'écrans et les chaînes les plus utilisés dans chaque session.
2. **Des attributs dynamiques :** qui permettent de comprendre la logique séquentielle dans laquelle l'utilisateur ordonne les actions qu'il effectue, par exemple :
   - La première et la dernière opération qu'il effectue dans la session. Cet attribut permet de faire la différence par exemple entre ceux qui veillent à fermer la session avant de quitter et ceux qui ne la ferment pas.
   - Quelle action est-elle faite souvent après 'action1' : par exemple, une fois une page ouverte, un utilisateur se dirige souvent vers un dialogue, alors qu'une autre personne ouvre souvent un onglet.

Une fois qu'on définit les attributs qui seront utilisés pour la prédiction, on passe à élaborer un plan de travail pour construire le modèle. Dans un premier temps, on se limite à un petit nombre d'attributs statiques qu'on peut extraire par des simples traitements sur les données brutes, il s'agit ici des features suivants :

- Le navigateur utilisé.
- La durée de la session : donnée par la dernière valeur de la forme *'txxxx'* dans la session.
- Le nombre d'occurrences de chaque action parmi toutes les actions possibles sur le logiciel. La liste des actions est obtenue en appliquant un filtre - qui élimine les écrans, les configurations et les chaînes utilisées - à toutes les actions présentes dans les données brutes. On compte au total 35 actions du type : "Clic sur une checkbox", "Affichage d'une erreur", "Filtrage / Tri", "Affichage d'une dialogue"...

— Le nombre total d'actions effectuées lors de la session, qui est donné simplement par la somme des occurrences de toutes les actions.

Ces attributs sont ajoutés à un nouveau DataFrame qui sera utilisé pour faire la prédiction, on fait ce choix pour éviter de déformer les données brutes.

On fait des simples statistiques sur trois attributs pour avoir une idée sur leur distribution :

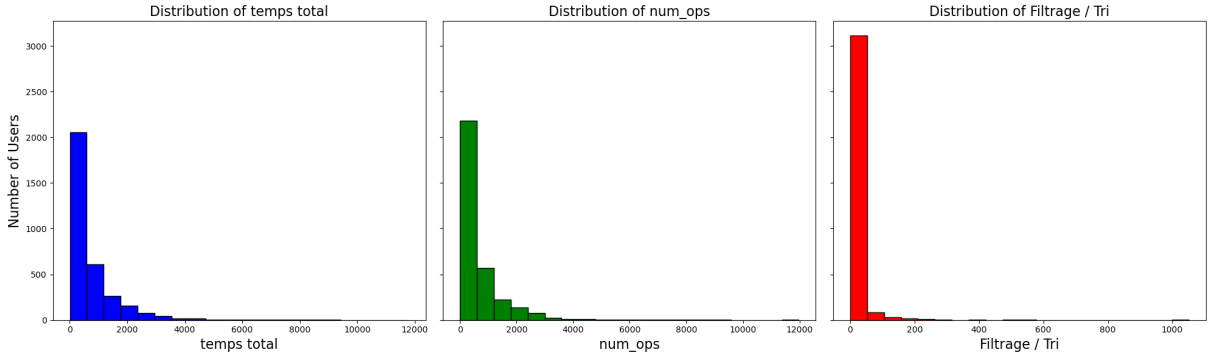


FIGURE 2 – Statistiques sur les attributs statiques

Les statistiques montrent que, pour les trois attributs analysés, la majorité des utilisateurs se concentrent autour de valeurs faibles, avec une distribution très asymétrique. Cela suggère que les attributs définis précédemment ne permettent pas de faire suffisamment de distinction entre eux. Par conséquent, les attributs dynamiques doivent aussi être utilisés pour avoir un modèle robuste, vu qu'ils permettent d'identifier les schémas typiques d'utilisation et de comprendre le flux d'action de chaque utilisateur.

Les deux premiers attributs dynamiques envisagés correspondent respectivement à la première et à la dernière action réalisée au cours de chaque session. Ensuite, on passe à analyser la fréquence des transitions d'une action à l'autre dans chaque session, par exemple la fréquence de la transition "Sélection d'un onglet $\longrightarrow$ Lancement d'une stat". Vu que le nombre de transitions possibles est élevé ($C_2^{35} = 595$), il est difficile de les ajouter manuellement, alors on utilise le module `TfidfVectorizer`, une méthode de transformation de texte en vecteurs numériques, qui permet de convertir une séquence d'actions *string* en une matrice de fréquences que les algorithmes de machine learning peuvent comprendre.

Une fois ces attributs tous ajoutés, on passe à inspecter la corrélation entre les attributs. Comme il peut être difficile d'inspecter les données lorsque on a un grand nombre de features, on peut commencer par tracer des graphiques de corrélation pour une ou quelques cibles particulières. On choisit d'inspecter la corrélation entre les attributs statiques. On ne fait pas cette inspection sur les attributs dynamiques car chaque attribut correspond à une transition distincte et représente son importance dans la session, alors il sont censés être indépendants par construction, et en plus leur corrélation est difficile à visualiser à cause de leur nombre élevé.

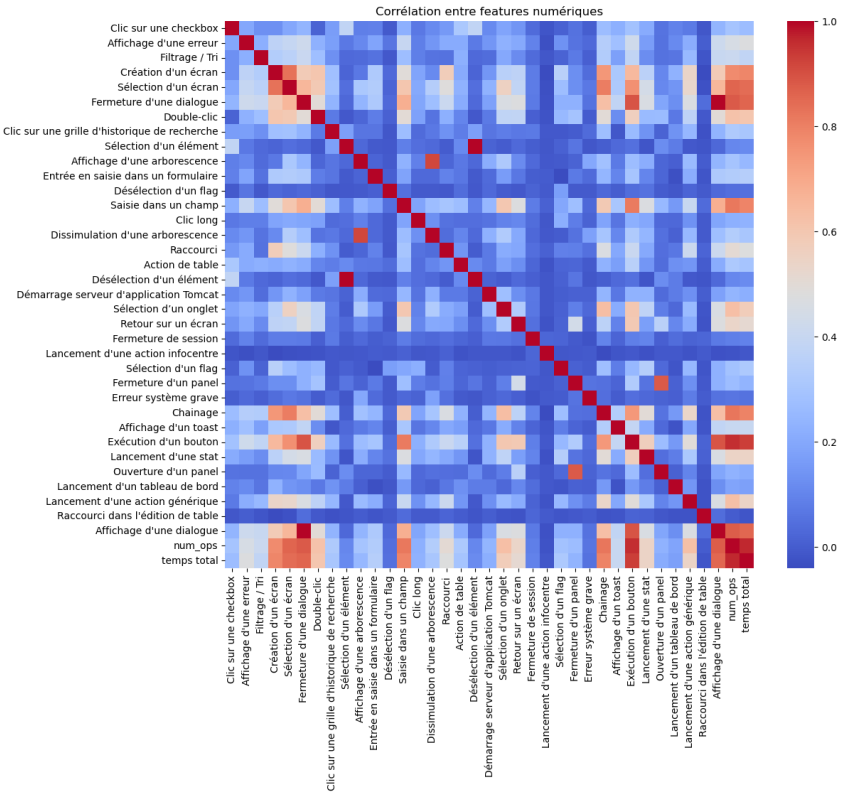


FIGURE 3 – Corrélation entre les attributs statiques

A travers la visualisation de la matrice de corrélation, on comprend qu'il y a des features à éliminer car ils présentent une haute corrélation avec d'autres features :

— Dé-sélection d'un élément.
— Fermeture d'une dialogue.
— Nombre d'opérations par session .

4 Développement d'une solution de classification automatique :

Une fois qu'on a exploré les données et mieux compris comment résoudre la tâche, c'est le moment de commencer à développer la solution de classification automatique supervisée.

La première opération consiste à encoder les attributs admettant des valeurs non numériques, on opte pour le **codage catégoriel**, on fait ce choix parce qu'il permet de conserver le même nombre d'attributs, ce qui permettra une meilleure visualisation de la corrélation et évitera des tableaux avec beaucoup de colonnes. Puis on sépare le jeu de données en deux sous-ensembles : un jeu d'entraînement et un jeu de validation. Le jeu de validation aidera dans la sélection du meilleur modèle. On choisit dans un premier temps les valeurs standard : 80% pour l'entraînement et 20% pour la validation. On choisit 4 algorithmes, et pour chaque modèle, on veille à varier les hyper-paramètres et à évaluer leur influence .Les algorithmes testés sont les suivants :

1. **Random Forest (RFC) :** Un modèle robuste face à la haute dimension et

excellent pour capturer les interactions non linéaires, on choisit comme hyperparamètres :
- `n_estimators=100` puis `300` : on commence à 100 comme référence. L'augmentation à 300 montre une amélioration de la performance, confirmant que le modèle bénéficiait de plus d'arbres pour stabiliser sa prédiction.
- `max_depth=None` : On laisse les arbres croître pleinement pour éviter de brider la découverte de micro-comportements.

2. **Machines à Vecteurs de Support (SVM) :** Testées pour leur efficacité théorique en haute dimension, elles se sont révélées inadaptées à nos données.
   - `class_weight='balanced'` : un paramètre essentiel pour ne pas ignorer les utilisateurs minoritaires (avec peu de sessions).
   - `kernel='linear', 'poly', 'rbf'` : On teste ces noyaux pour évaluer différentes frontières de décision.
   - Paramètre `C` choisi (valeurs testées 7 et 13) grand pour pénaliser les erreurs et laisser une petite marge entre les classes.
3. **Régression Logistique :** Utilisée comme modèle de base, son défaut principal c'est qu'il prend beaucoup de temps.
   - `multi_class='multinomial'` : Requis pour un problème à 247 classes ; car gère toutes les classes simultanément. Par contre, le mode `'ovr'` serait peu efficace et peu optimal.
   - `max_iter=1000` car le solveur `'lbfgs'` nécessite ce nombre d'itérations pour converger lorsque le nombre des features est presque 1000.
4. **Réseau de Neurones (MLP) :** Testé pour capturer des relations non linéaires profondes :
   - `hidden_layer_sizes=(100, 50)` : Une architecture standard qui compresse les nombreux features en une représentation dense.
   - `hidden_layer_sizes=(200, 70)` : Testé car le premier score n'était pas satisfaisant (détaillé en section 5) pour voir si plus de capacité (neurones) résoudrait un sous-apprentissage, mais l'amélioration fut marginale.
   - `early_stopping=True` : pour éviter le sur-apprentissage et réduire le temps d'entraînement.

5 Métriques d'Évaluation

Pour évaluer la performance de nos modèles, la métrique principale retenue est le **F1-Score (Macro)**.

Dans un problème de classification multi-classe avec 247 classes et un jeu de données déséquilibré (certains utilisateurs ont beaucoup plus de sessions que d'autres), la précision simple (accuracy) n'est pas une mesure fiable.

Le F1-Score Macro calcule le score F1 pour chaque classe indépendamment, puis effectue la moyenne de ces scores. Cela garantit que chaque utilisateur, qu'il ait 4 ou 75 sessions, pèse autant dans l'évaluation finale. On utilisera également la précision (accuracy) lors de la validation croisée pour confirmer la robustesse globale du modèle.

6 Résultats des Modèles

Les résultats obtenus sur l'ensemble de validation sont présentés dans le tableau 1.

Modèle	Hyperparamètres (simplifiés)	F1-Score (Macro)
Random Forest	`n_estimators=300`	**0.7458**
Random Forest	`n_estimators=100`	0.7298
Régression Logistique	`multinomial, lbfgs`	0.5820
MLP Classifier	`(200, 70), adam`	0.5101
MLP Classifier	`(100, 50), adam`	0.5080
Linear SVC	`C=7.0`	0.3683
RBF SVC	`C=13.0`	0.1883
Poly SVC	`C=13.0`	0.1234

TABLE 1 – Résultats de la première phase de modélisation.

L'analyse de ces résultats laisse déduire que :

— Le **Random Forest Classifier (RFC)** surpasse largement tous les autres modèles.

— Les SVM à noyaux (`Poly`, `RBF`) s'effondrent, probablement à cause de la haute dimensionnalité des données.

— Le RFC bénéficie de l'augmentation du nombre d'arbres (0.7458 pour 300 contre 0.7298 pour 100), indiquant qu'un modèle plus complexe capture mieux le signal.

— Le comportement utilisateur est défini par des interactions de features non linéaires, faisant du Random Forest le candidat le plus probable au succès.

Une validation croisée (K-Fold, K=3) sur le RFC (`n_estimators=100`) confirme cette performance, donnant une précision moyenne de **0.7627**. En plus, le modèle RFC présente le bénéfice de la rapidité lors de son entraînement.

7 Amélioration de la solution

Suite à l'exploration des différents algorithmes de machine learning, le choix final s'installe sur le Random Forest Classifier. Mais, on n'est pas encore satisfait du score obtenu. On suspecte que la part de l'information perdue est à cause du fait qu'on ignore le contexte des actions : les écrans, les configurations d'écrans et les chaînes utilisées. On mène alors une deuxième phase de features engineering où on capture les "patterns" contenues dans les actions brutes. Vu leur nombre élevé, on se sert cette fois encore du module `TfidfVectorizer` qui mesure leurs fréquences dans toutes les sessions. Le jeu de données final combine les features statiques, les bigrammes représentant les transitions entre les actions (968 features) et les fréquences des patterns (1000 features), pour un total de 1943 caractéristiques.

En ré-entraînant notre meilleur modèle (RFC, `n_estimators=300`) sur ce jeu de données enrichi, les résultats sont les suivants :

— **F1-Score (Macro)** sur split 80/20 : 0.8780.

— **Précision** en validation croisée $K = 3$: 0.8716.

l'amélioration observée confirme que l'identification des modules et configurations spécifiques est un facteur clé pour identifier un utilisateur.

8 Identification des Variables Influentes

L'évolution des performances lors de notre démarche itérative permet d'inférer quelles caractéristiques sont les plus discriminantes pour l'identification d'un utilisateur.

1. **Patterns TF-IDF (Influence la plus élevée) :** Le saut de performance de 0.746 à 0.878 est du à l'ajout des features de patterns. alors, ce que l'utilisateur consulte est un marqueur comportemental puissant.
2. **Bigrammes d'Actions TF-IDF :** La supériorité du RFC (0.74) sur la Régression Logistique (0.58) en Phase 1 montre que les features de transition entre les actions sont fondamentaux.
3. **Les attributs statiques :** fournissent des indicateurs comportementaux simples mais efficaces.

9 Discussion

L'approche itérative adoptée a démontré que si le comptage d'actions fournit une base, le véritable pouvoir prédictif réside dans le contexte (les patterns) et la séquence (les bigrammes). Le modèle **Random Forest Classifier** a prouvé son efficacité à gérer cet espace de caractéristiques de grande dimension et complexes. Sa nature ensembliste est particulièrement adaptée, car elle gère nativement la haute dimensionnalité sans besoin de normalisation (feature scaling), (contrairement aux SVM ou MLP) et excelle à modéliser les interactions non linéaires complexes entre les features.

Atteindre un F1-Score Macro de 0.899 sur le test de prédiction est un résultat satisfaisant pour un problème de classification à 247 classes, démontrant que les utilisateurs de "Copilote" ont des signatures comportementales uniques et identifiables.

10 Conclusion

En guise de conclusion, le pipeline de classification développé est complet et performant pour l'identification d'utilisateurs. L'analyse des caractéristiques étudiées montre que le comportement des utilisateurs est davantage lié aux outils qu'ils utilisent et à la dynamique de leurs interactions qu'à leurs caractéristiques statiques. Dans le cas d'une base de donnée ayant de telles propriétés, le modèle final retenu est un **Random Forest Classifier**, entraîné sur un jeu de 1943 caractéristiques hybrides, vu qu'il présente mieux de performances par rapport aux autres modèles. Effectivement, il permet de reconnaître 90% environ des utilisateurs à partir de leurs traces, et donc forme un bon outil pour diagnostiquer des tentatives d'intrusion logicielle, par exemple lorsqu'un pirate tente de se faire passer pour un utilisateur réel en se connectant avec son compte.

Référence du Code Source

L'ensemble du code et le notebook Jupyter (`Notebook DS.ipynb`) développés pour cette analyse sont disponibles publiquement sur le dépôt GitHub du projet : https://github.com/OSEHIT/Copilot$_{U}sers_{C}lassifier$